# Perceptron (neural) branch predictor

Daniel A. Jiménez

Slides adapted by: Dr Sparsh Mittal

# Can We Do Better?

- Last-time and 2-bit counter predictors exploit “last-time” predictability. Can we do better?

- Idea: A branch’s outcome can be correlated with other branches’ outcomes
  - Global branch correlation

# Global Branch Correlation (I)

- Recently executed branch outcomes in the execution path is correlated with the outcome of the next branch

```
if (cond1)
...
if (cond1 AND cond2)
```

- If first branch not taken, second also not taken

```
branch Y: if (cond1) a = 2;
...
branch X: if (a == 0)
```

- If first branch taken, second definitely not taken

# Global Branch Correlation (II)

```
branch Y: if (cond1)
...
branch Z: if (cond2)
...
branch X: if (cond1 AND cond2)
```

- If Y and Z both taken, then X also taken
- If Y or Z not taken, then X also not taken

# Global Branch Correlation (III)

```
if (aa==2)                    // B1
      aa=0;
if (bb==2)                    // B2
      bb=0;
if (aa!=bb) {                 // B3
      ....
   }
```

If B1 is true (i.e., aa==0@B3) and B2 is true (i.e. bb=0@B3) then B3 is certainly false

# Conditional Branch Prediction is a Machine Learning Problem

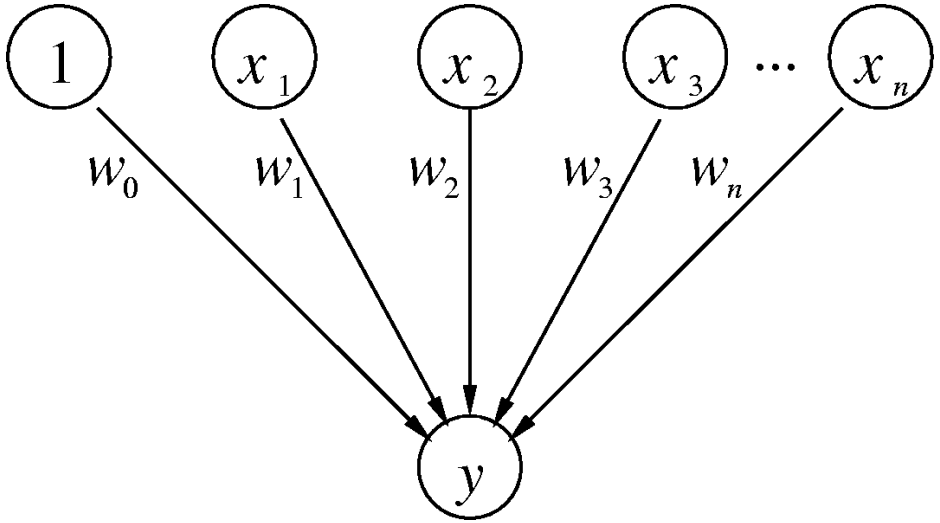
- The machine learns to predict conditional branches
- So why not apply a machine learning algorithm?
- Artificial neural networks
  - Simple model of neural networks in brain cells
  - Learn to recognize and classify patterns
- Idea: Use fast and accurate perceptrons [Rosenblatt `62, Block `62] for dynamic branch prediction

# Input and Output of the Perceptron

- The inputs to the perceptron are branch outcome histories
  - Can be global or local (per-branch) or both
  - Conceptually, branch outcomes are represented as
    - +1, for taken
    - -1, for not taken
- The output of the perceptron is
  - Non-negative, if the branch is predicted taken
  - Negative, if the branch is predicted not taken
- Ideally, each static branch is allocated its own perceptron

# Branch-Predicting Perceptron

- Inputs ($x$'s) are from branch history and are -1 or +1
- $n$ + 1 small integer weights ($w$'s) learned by on-line training
- Output ($y$) is dot product of $x$'s and $w$'s; predict taken if $y \geq 0$
- Training finds correlations between history and outcome




$$y = w_0 + \sum_{i=1}^{n} x_i w_i$$

# Training Algorithm

$x_{1..n}$ is the $n$-bit history register, $x_0$ is 1.
$w_{0..n}$ is the weights vector.
$t$ is the Boolean branch outcome.
$\theta$ is the training threshold.

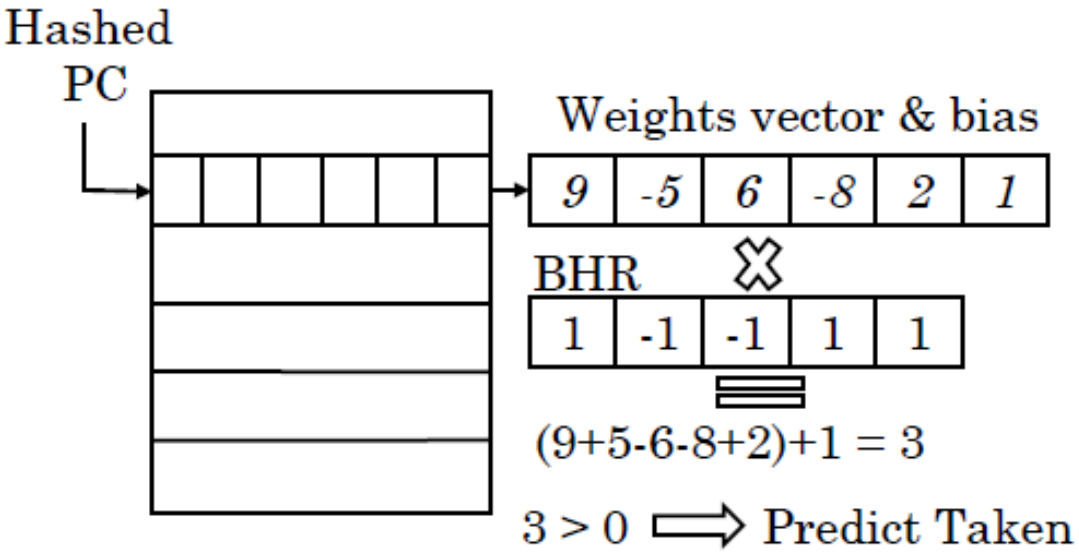
```
if |y| ≤ θ or ((y ≥ 0) ≠ t) then
    for each 0 ≤ i ≤ n in parallel
        if t = x_i then
            w_i := w_i + 1
        else
            w_i := w_i − 1
        end if
    end for
end if
```

For our course, we ignore this condition => we will always retrain the predictor after each prediction

# What Do The Weights Mean?

- The bias weight, $w_0$:
  - Proportional to the probability that the branch is taken
  - Doesn't take into account other branches
- The correlating weights, $w_1$ through $w_n$:
  - $w_i$ is proportional to the probability that the predicted branch agrees with the $i^{\text{th}}$ branch in the history
- The dot product of the $w$'s and $x$'s
  - $w_i \times x_i$ is proportional to the probability that the predicted branch is taken based on the correlation between this branch and the $i^{\text{th}}$ branch
  - Sum takes into account all estimated probabilities
- What's $\theta$?
  - Keeps from overtraining; adapt quickly to changing behavior

# Example: (1 of 3)




(a) First time prediction

# Example: (2 of 3)

**If outcome = not taken**

Weights vector & bias

| | | | | | |
|---|---|---|---|---|---|
| *9* | *-5* | *6* | *-8* | *2* | *1* |

⇩

| | | | | | |
|---|---|---|---|---|---|
| *8* | *-4* | *7* | *-9* | *1* | *0* |
| -1 | +1 | +1 | -1 | -1 | -1 |

BHR

| | | | | |
|---|---|---|---|---|
| 1 | -1 | -1 | 1 | 1 |

(b) Retraining

# Example: (3 of 3)

Weights vector & bias

| | | | | | |
|---|---|---|---|---|---|
| *8* | *-4* | *7* | *-9* | *1* | *0* |

$\times$

| | | | | |
|---|---|---|---|---|
| 1 | -1 | -1 | 1 | 1 |

$=$

$(8+4-7-9+1)+0 = -3$

$-3 < 0 \Longrightarrow$ Predict Not Taken

(c) Next time prediction

# Example 2 and Solution

- Assume a neural predictor entry has the weight vector as
- [10, 4, -9, -2, 2] and branch history register as
- [1, -1, 1, -1]. Bias weight is the last weight. Find its prediction.
- If the outcome is not-taken, show updated weight vector. Find second prediction.
- Solution: Product is 10-4-9+2 +2 =1, so prediction is taken.
- It is incorrect, so new weight: [9 5 -10 -1 1]
- New product is 9-5-10+1+1, which is negative, so prediction is not-taken.

# Example 3 and solution

- Assume an entry has the weight vector as [10, 4, -9, -2, 2]
- Branch history register is [1, -1, 1, 1]. Bias weight is last weight.
- Find its prediction. If outcome is taken, find updated weight vector.
- Then, find the second prediction.
- **For first prediction, the sum of product of weight and BHR is:**
- **10-4-9-2+2 = -3 which is less than zero. So the prediction is "not taken".**
- **This is incorrect.**
- **So, we change the weight vector to [11 3 -8 -1 3].**
- **The second prediction is taken since : 11-3-8-1+3 is more than zero.**